

Python Programming

Overview

Python is a high-level, interpreted, general-purpose programming language known for its readable syntax and large standard library. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming, which makes it suitable for scripting, web development, data analysis, automation, and artificial intelligence.

Core Data Types

- Numeric types: int, float, and complex for numerical computation.
- Sequence types: str, list, and tuple for ordered collections of values.
- Mapping type: dict for key-value pair storage and fast lookups.
- Set types: set and frozenset for unordered collections of unique elements.
- Boolean type: bool, representing True or False values.

Control Flow

Python uses indentation rather than braces to define code blocks, which enforces consistent formatting across a codebase. Conditional logic is expressed with if, elif, and else statements, while iteration is handled through for loops over iterables and while loops that repeat based on a condition. The language also supports comprehensions, which build lists, sets, or dictionaries in a single, concise expression.

Functions and Scope

Functions are defined with the def keyword and can accept positional arguments, keyword arguments, default values, and variable-length argument lists using *args and **kwargs. Python resolves variable names using the LEGB rule, checking local, enclosing, global, and finally built-in scopes in that order.

Object-Oriented Programming

Classes in Python are defined with the class keyword and support single and multiple inheritance. Special methods, often called dunder methods, allow custom classes to integrate with built-in behavior such as string representation, equality comparison, and iteration. Instance attributes are typically set inside the __init__ constructor method.

Modules and Packages

A module is simply a Python file, and a package is a directory containing an `__init__.py` file along with related modules. The import statement loads modules into the current namespace, and virtual environments are commonly used to isolate a project's dependencies from the system-wide Python installation.

Error Handling

Python uses `try`, `except`, `else`, and `finally` blocks to handle exceptions. Custom exception classes can be created by subclassing the built-in `Exception` class, which allows an application to define its own meaningful error types for specific failure conditions.

Asynchronous Programming

The `async` and `await` keywords, combined with the `asyncio` library, allow Python programs to perform non-blocking I/O operations such as network requests or database queries. This is especially relevant when working with asynchronous web frameworks such as `FastAPI`, where request handlers can be declared as coroutine functions to avoid blocking the event loop.

Popular Libraries

- `requests` and `httpx` for making HTTP calls.
- `pandas` and `numpy` for data manipulation and numerical computing.
- `pytest` for writing and running automated tests.
- `SQLAlchemy` for interacting with relational databases through an ORM.